Time-Series Forecasting

> Part of the Data Science / ML curriculum. This document covers Exponential Smoothing (ETS, specifically Holt's Damped Trend), multi-step recursive forecasting, what-if scenario modeling, and how this family of models relates to the regression models covered previously and to other common forecasting approaches.

1. How this differs from the regression models

The models in Regression Models predict a target from a table of *engineered features* — they have no inherent notion of "time" beyond whatever time-based features you built. **Exponential smoothing models** are a genuinely different approach: they're **state-space models** that forecast a series using only its own past values, with no external feature matrix at all. This makes them a natural fit whenever you specifically want to forecast a series' own future based on its own historical pattern, rather than explain it in terms of other variables.

2. Concept: ETS(Error, Trend, Seasonal)

The general family is denoted **ETS(Error, Trend, Seasonal)**, where each component can take one of a few standard forms:

Component	Options	Meaning
Error	Additive (A) or Multiplicative (M)	whether forecast errors are added to the state update, or scale multiplicatively with the series' level
Trend	None (N), Additive (A), Additive **Damped** (Ad)	whether/how the series' trend is modeled — undamped grows a constant amount each period; damped decays that growth toward flat
Seasonal	None (N), Additive (A), Multiplicative (M)	whether a repeating within-cycle pattern (e.g., a monthly or weekly cycle) is modeled

For **annual, non-seasonal data**, the seasonal component is simply `None` — there's no within-year cycle to model at that frequency. The variant most relevant to this kind of project is **ETS(A, Ad, N)**, also known as **Holt's Damped Trend**.

2.1 Why damping matters

An *undamped* trend model extrapolates whatever rate of change it last observed **indefinitely** — over a long forecast horizon this becomes physically implausible for almost any real quantity (unbounded growth, or a decline that runs through zero into negative territory). A **damped** trend instead lets each period's trend increment shrink geometrically toward zero, controlled by a damping parameter $0 < \varphi < 1$ — the forecast keeps moving in the same direction but at a decelerating rate, which is a far more realistic default assumption for most real-world series over a multi-year horizon.

2.2 When to prefer ETS over ARIMA

ARIMA (AutoRegressive Integrated Moving Average) is the other classical workhorse for time-series forecasting, and it's worth understanding the trade-off rather than treating one as strictly better:

	ETS	ARIMA
Modeling decisions required	Trend/seasonal form (a handful of standard combinations)	Requires choosing the order of differencing, autoregressive, and moving-average terms (`p`, `d`, `q`), typically via ACF/PACF plots or an automated search
Stationarity requirement	Not required upfront — the model itself handles trend	Typically requires the series to be made stationary first (via differencing) — see §5
Long-horizon behavior	The damped variant explicitly bounds extrapolation	An undifferenced or unit-root model can extrapolate a trend indefinitely, which can become implausible at long horizons unless deliberately constrained
Data volume needed	Works reasonably well with a fairly small number of observations (a few dozen)	Also works with modest data, but order selection becomes less reliable with very few observations

ETS's damped trend variant is the better default whenever you specifically need a **long-horizon** forecast and want built-in protection against implausible unbounded extrapolation, with fewer upfront modeling decisions to get right. Other tools worth knowing exist for context: **Prophet** (originally from Meta) automates a lot of the trend/seasonality decomposition and handles holidays/multiple seasonalities well, at the cost of being a heavier, more opinionated dependency than statsmodels; **STL decomposition** (Seasonal-Trend decomposition using LOESS) is a general-purpose tool for *visualizing* and separating a series into trend/seasonal/residual components, useful for understanding a series even if you ultimately forecast it with ETS or ARIMA rather than STL directly.

3. Fitting the model

```
from statsmodels.tsa.holtwinters import ExponentialSmoothing

model = ExponentialSmoothing(
    train_series,        # a pandas Series, indexed by time (integer period or datetime)
    trend='add',          # additive trend
    damped_trend=True,    # enables the damping parameter phi
    seasonal=None,        # no seasonal component for non-seasonal data
)
fit = model.fit(optimized=True)   # estimates alpha, beta*, phi (and the initial level/trend) by
        maximizing fit
```

`ExponentialSmoothing`'s other relevant constructor arguments: `seasonal_periods` (required if `seasonal` is set to `'add'` or `'mul'` — the length of one seasonal cycle, e.g., `12` for monthly data with an annual cycle); `initialization_method` (how the model's initial level/trend/seasonal state is estimated — `'estimated'` is a sensible default).

3.1 Interpreting the fitted parameters

Three fitted parameters are worth being able to interpret at a glance:

- **α (smoothing level, `fit.params['smoothing_level']`)** — how much weight the most recent observation gets versus the smoothed history. High α → the level estimate reacts quickly to recent data ("forgets" older observations fast); low α → the level is close to a long-run average.
- **β* (smoothing trend, `fit.params['smoothing_trend']`)** — how quickly the *trend estimate itself* updates in response to recent acceleration/deceleration. High β* → the trend reacts fast to recent changes in slope.
- **ϕ (damping, `fit.params['damping_trend']`)** — how fast the trend decays toward flat over the forecast horizon. ϕ near 1 → close to an undamped, persistent trend; ϕ near 0 → the trend fades quickly and the forecast flattens out. Since the damping compounds geometrically, even a ϕ fairly close to 1 (e.g., 0.97) produces meaningful flattening over a couple of decades (`0.97^25 ≈ 0.47`, meaning the trend's contribution has roughly halved by 25 periods out).

4. Forecasting with confidence intervals

`HoltWintersResults` (the object `fit()` returns) has a simple point-forecast method, and a simulation-based way to get confidence intervals:

```
steps = N_PERIODS_AHEAD
point_forecast = fit.forecast(steps)   # a pd.Series, index continuing on from the last training
        period


# Confidence intervals via simulation (no closed-form get_forecast() for this model class)
sim = fit.simulate(nsimulations=steps, repetitions=1000, error='add')
ci_lower = sim.quantile(0.025, axis=1)
ci_upper = sim.quantile(0.975, axis=1)
```

`simulate()` generates many possible future paths consistent with the fitted model's estimated error distribution (`repetitions` of them), and taking quantiles across those paths at each future period gives an empirical confidence interval — necessary here because, unlike some other time-series model classes, `HoltWintersResults` doesn't expose a closed-form `get_forecast()` with built-in analytic confidence intervals.

5. Multi-step recursive forecasting, and why model choice matters here

Both a regression model (Regression Models, §3) and an ETS model can, in principle, be extended into a **recursive (iterative) multi-step forecast**: predict one step ahead, feed that prediction back in as the next step's lag/input value, and repeat. This is where Linear Regression's unbounded extrapolation (discussed in Regression Models, §3.5) becomes a real practical problem: if a linear model predicts a value below the target's physically meaningful range (e.g., a negative value for something that can't be negative), and that value feeds back in as a lag feature, subsequent predictions can diverge rapidly, compounding the error at every step.

Random Forest predictions, by contrast, are **bounded by the range of values seen during training** — a forest can only output values close to what its trees' leaves were trained on; it cannot extrapolate outside that range the way a linear model can. This makes Random Forest considerably more stable for long-horizon recursive forecasting, even though a simpler model might score better on a short-horizon holdout evaluation. **The right model for a 1-step-ahead evaluation is not automatically the right model for a long-horizon recursive forecast** — evaluate each use case on its own terms, and prefer a model whose *failure mode* at the horizon you actually care about is acceptable, not just whichever model wins on a single aggregate metric.

6. Stationarity — a deeper look

Feature Engineering, §2.3 introduced **stationarity** informally as the assumption that a series' statistical properties (its level, trend, variability) don't change over time. This matters more formally for ARIMA-style models, which typically require the series to be stationary (or made stationary via differencing) before fitting — ETS models handle a trend explicitly as part of the model itself, which is part of why they need less upfront stationarity work.

To check formally rather than just visually:

```
from statsmodels.tsa.stattools import adfuller

result = adfuller(series)
adf_statistic, p_value = result[0], result[1]
# a small p-value (conventionally < 0.05) suggests the series IS stationary
# (rejects the null hypothesis of a "unit root" / non-stationarity)
```

The **Augmented Dickey-Fuller (ADF) test** is the standard statistical test for this: its null hypothesis is that the series is non-stationary (has a "unit root"); a small p-value lets you reject that null and treat the series as (likely) stationary. In practice, for the kind of annual, slowly-trending data this curriculum focuses on, most raw series will fail this test (they have an obvious trend) — which is exactly why a **trend-modeling** approach like ETS, which doesn't require pre-differencing, is a more direct fit than an ARIMA model that would first need that trend differenced out.

7. What-if scenario modeling

A common extension once a baseline forecast exists is to construct alternative "what if" scenarios by applying a compounding adjustment rate to the baseline:

```
import numpy as np

years_elapsed = np.arange(len(baseline_forecast))
scenario = baseline_forecast * (1 - rate) ** years_elapsed   # e.g. rate=0.02 for a 2%/year
        compounding reduction
```

This is useful for illustrating bounding cases (e.g., "what if this rate of change were 2× faster or slower"), but it's important to be explicit about its limitation: a single flat rate applied uniformly across very different segments

(entities with very different starting points, growth rates, or structural conditions) is a simplification, not a causal model. Report and interpret this kind of scenario as an illustrative, order-of-magnitude comparison — not a calibrated prediction of what any specific segment would actually do under a real intervention.

Which baseline does a scenario start from? A scenario is "the baseline, adjusted," so it inherits the baseline's starting point. The forecast you fitted for *evaluation* (§9) stops at the training cutoff so later years can be held out — which means a scenario built on it starts from a level that never saw the held-out years. In one real case that made the scenarios begin 3.7% above the most recent observation in aggregate, and by up to ±50% for individual entities. The fix is **two artifacts, not a moved cutoff**: leave the evaluation fit alone (it still does its teaching job), and fit the same model a second time on *every* observed year for scenario use. Then check the seam between the last observation and the first projected year in aggregate and per entity (Multi-Source Pipelines, §8), and flag — don't silently smooth — a single anomalous year at the start of a training window, which can force an extreme smoothing parameter for that one entity.

8. Core libraries summary

Library	Used for
statsmodels	`statsmodels.tsa.holtwinters.ExponentialSmoothing`, `statsmodels.tsa.stattools.adfuller`
pandas	time-indexed `Series` construction, the recursive-forecast bookkeeping
NumPy	compounding-rate scenario calculations
(for context, not covered in depth here) **Prophet**, **pmdarima** (automated ARIMA order selection)	alternative forecasting tools worth knowing exist

9. Testing and validating forecasting code

- **Evaluate on a genuine holdout**, exactly as in Regression Models, §6.1 — fit on data up to a cutoff, forecast forward, and compare against actual values the model never saw. Report the same MAE/RMSE metrics (Regression Models, §6.3) so ETS results are directly comparable to the regression models' results on the same holdout.
- **Sanity-check the fitted parameters (§3.1)** against domain intuition — a damping parameter or trend estimate that doesn't make sense for the series you're looking at is worth investigating before trusting the resulting forecast.
- **Stress-test recursive forecasts (§5) at the actual horizon you plan to use**, not just at the horizon you happen to have holdout data for — a model that looks fine 5 steps out can behave very differently 25 steps out, and the only way to know is to actually run the recursion that far and inspect it (including plotting it) rather than assuming short-horizon accuracy generalizes.

See also

- Data Science / ML index
- Regression Models — the alternative, feature-based approach to predicting the same kind of data
- Python API Backend for how to serve these forecasts (and their confidence intervals) over HTTP